

Pi Stuff

现在有什么，还缺什么

把已经能用的、仍需补齐的、等待上游的和明确不做的内容分开说清楚。

| 目录

01	先说结论	3
02	现在打开 Pi, 会得到什么	4
03	目前的 UI 到了哪一步	5
04	六项已经能用的能力	7
05	是否能用、好用、可靠	9
06	我们约定的, 还有什么没做	11
07	明确不做, 不算欠账	14
08	建议的下一步	15
A	依据与术语	17

先说结论

Pi Stuff 的第一阶段已经完成：它不再是一堆想法，也不再只是原型。现在本机启动 Pi，会自动加载一套统一的日用界面，以及 Tools、Agents、Todo、BTW 和 Permissions。它已经可以拿来日常写代码。

一句话判断

当前版本已经是“好用的 Pi 日用基础层”，但还不是我们最初设想的“完整成熟 coding agent 能力套件”。

判断	结论	人话解释
能用	是	当前 Pi 设置已经指向本地 Aggregate Package；重启 Pi 后会直接加载。
好用	基础体验已明显成型	工具输出、实时思考、状态栏、子 Agent、Todo 和临时页面已经遵守同一套 UI 规则。
可靠	对已交付范围，是	76 个测试文件、453 个声明测试，以及真实 Pi、窄屏、session resume、打包和子 Agent 路径均已验证。
已发布	否	七个 package 仍是 0.0.0，只能从本地仓库安装，尚不能从 npm 正式安装。
能力完整	否	长期记忆、Plan、后台命令管理、Goal、Loop、恢复、Web 和 MCP 还没有生产实现。

最关键的状态划分

状态	内容
现在已经能用	统一 UI、Tools、Permissions、Agents、Todo、BTW。
已经约定，尚未做	Mem0 长期记忆，以及对未来能力路线的正式 Beads 归档。
方向已讨论，尚未正式排期	Plan、结构化提问、后台 shell/Monitor、Goal、Loop/Cron、rewind、Web、MCP、/doctor 和默认 Skills。
现在做不了，等待 Pi	相邻低风险 Tool 自动分组；BTW 完整走 Pi Host 的旁路模型调用链。
明确不做	LSP、浮动窗口、fork Pi、第二套 Host、默认内建可交互浏览器、跨 session Fleet/Agent Teams。

| 现在打开 Pi, 会得到什么

你不需要再手动拼装这些插件。当前 `~/.pi/agent/settings.json` 已经指向 `../../dev/pi-stuff/packages/pi-stuff`。新启动的 Pi 会依次加载 UI、Tools、Permissions、Agents、Todo 和 BTW。

普通一次 coding session 的样子

1. 启动: 看到简洁的 Welcome, 底部是已经讨论并实现的 Statusline; 没有 Pi Stuff 广告, 也没有第二套主界面。
2. 输入: 命令和 skill 会高亮; 在一段文字中输入 slash 也能得到真实命令补全。
3. 思考: 模型推理时只显示一行 `* thoughts:`, 同一个思考块原位增长, 新思考块替换旧块, 不再把多个阶段挤成一长行。
4. 调用工具: Read、Write、Edit、Bash 等操作各用一行紧凑显示; 运行、成功、失败或拒绝共用一个固定圆点位置, 不会跳动或错位。
5. 任务较长: Agent 可自动建立 Todo, 并把独立工作交给后台子 Agent; 主对话可以继续。
6. 想顺手问一句: 输入 `/btw` 问题, 得到一次旁路回答; 这段问答不进入主对话, 也不改变主 Agent 的上下文。
7. 需要看细节: `/tools`、`/agents`、`/btw`、`/permissions` 和 `/ui` 使用同一种全宽临时页面; 关闭后恢复原来的输入草稿和界面。
8. 有危险操作: 普通读写和测试不会询问; 明显可能毁掉工作区或删除项目外内容时, 系统会拒绝或要求仅批准这一次。

它仍然是 Pi, 不是一个新外壳

Pi 继续拥有: 模型、主 Agent 循环、session、编辑器、原生工具、Settings 和 TUI。
Pi Stuff 负责补充: 统一呈现、工作组织、旁路问答、子 Agent 与防手滑保护。

七个 package

Package	作用
@jczhang02/pi-stuff	唯一入口, 按固定顺序加载整套能力。
@jczhang02/pi-stuff-ui	Statusline、Welcome、实时 Thoughts、输入增强、 <code>/ui</code> 和共享临时页面。
@jczhang02/pi-stuff-tools	统一内建 Tool 和 Pi Stuff 自有 Tool 的显示。
@jczhang02/pi-stuff-permissions	默认安静的破坏性命令断路器。
@jczhang02/pi-stuff-agents	当前 session 内的前台和后台子 Agent。
@jczhang02/pi-stuff-todo	当前工作清单、依赖关系和 session 恢复。
@jczhang02/pi-stuff-btw	不污染主对话的一次性旁路问答。

目前的 UI 到了哪一步

UI 是这一阶段完成度最高的部分。总体原则已经从“每个插件各做各的”变成“所有 Pi Stuff 能力使用同一套信息层级、窄屏规则、颜色和临时页面”。Claude Code 是主要行为参考，但实现只使用 pi 的公开接口，不复制其代码。

正常对话界面

位置	现在的样子	为什么这样做
Welcome	启动时显示模型、目录和可用项目数量；宽屏双栏，窄屏自动堆叠或只保留身份信息。	先让用户知道“在哪、用什么”，不占常驻空间。
对话内 Tool	一项操作一行，固定圆点槽位；详情只在 /tools 中展开。	避免 Ctrl+O 一次展开整段历史造成卡顿。
实时 Thoughts	一行 * thoughts: ；只显示当前思考块，流式原位更新。	保留思考中的即时感，同时不让多个阶段堆叠成噪音。
Todo	编辑器上方最多五项和一条 overflow；没有任务时完全不占空间。	回答“当前计划是什么”，不变成第二个项目管理器。
Agent roster	编辑器下方一行一个 Agent；显示短任务名、状态和用时，完成后保留 30 秒。	回答“现在谁在工作”，详细任务和结果留在 /agents 。
Statusline	模型、thinking、目录、Git、Context、cache/cost，以及将来真正存在时的 Goal/MCP/Loadout 状态。	保留旧配置中熟悉的视觉语法，不重复 Agent、Todo、BTW、Permission 或 Tool 状态。

临时交互统一成一种页面

/ui、/tools、/agents、/btw 和 /permissions 都打开同一种全宽、非浮动 Command Dialog。它位于分隔线下方，临时接管输入区域；Esc 返回或关闭，关闭后恢复输入草稿、footer、working row、Todo 和 Agent roster。

这也是已冻结的整体 UI 理念：不做居中卡片，不做遮挡对话的浮窗，不做每个插件自己的弹窗体系。设置类内容继续使用 Pi 原生 SettingsList；Pi Stuff 只负责把它放进共同容器。

/ui 现在能调什么

设置	效果
Statusline	显示或隐藏底部状态信息。
Statusline density	自动、完整或紧凑。
Latest prompt	空间允许时显示最近一次用户请求。
Statusline icons	自动判断 Nerd Font，或强制 Nerd/ASCII。
Welcome header	下一次启动时显示或隐藏 Welcome。
Input highlighting	高亮真实命令与 skills。
Inline slash autocomplete	在输入中的 slash 后给出补全。
Tool running timer	长时间 Tool 运行时显示用时。

窄屏不再硬挤

Tool、Agent、Todo、Permission、Statusline 和 Welcome 已统一审计 100、64、48、32、24 列宽，并覆盖中文、emoji、长路径和复杂 Unicode。空间不足时，先整体去掉低价值字段，再保留身份和状态；不会出现 `sample.tx...done` 这种省略号与状态粘成一个词的情况。

仍然没有全面重写的地方

普通 assistant 正文、一般 diff、测试日志和错误文本大多仍由 Pi 原生 transcript 渲染。Pi Stuff 已接管的是 Thoughts、Tools、Agent 详情和自有能力界面，不应宣称已经重做了所有对话输出。

| 六项已经能用的能力

1. Tools: 统一显示, 不改工具本身

Pi 的七个内建工具已经使用同一套紧凑显示。运行时圆点在原位置闪烁, 结束后用语义色表示成功或异常; 路径和结果状态在窄屏下仍保持清楚。/tools 可以打开最近操作, 逐项查看最多 240 行、24 KiB 的细节。

这层只改显示, 不改 schema、执行、权限、结果内容或模型看到的信息。Todo 和 subagent 等 Pi Stuff 自有工具也复用这套 renderer。此前 session resume 后历史 Tool 恢复成 raw 样式的问题已经修复。

真实限制: 当前仍是一项 Tool 一行。相邻的 Read/Search 自动分组要等待 Pi 提供公开 transcript grouping 接口。

2. Agents: 把独立工作交给子 Agent

主 Agent 可以自动启动单个或并行子 Agent, 默认放到后台。每个 Agent 有稳定身份、独立 transcript, 可以 steer、stop、resume 和查看状态; 支持最多三层嵌套、20 个同时运行、每个 session 200 次启动, 并可选择 Git worktree 隔离。

roster 使用短 description, 真正执行仍拿到完整 task。完成行保留 30 秒, 也可以提前隐藏; 隐藏只影响下方列表, /agents 中的任务、结果和 transcript 仍然存在。子 Agent 内的 Tool 结果能够对应到具体调用, 危险命令会回到根 Pi 请求处理。

真实限制: 它只管理当前 session, 没有 Fleet、Agent Teams、跨 session 队伍、saved chains、scheduler、watchdog、memory 或 sharing。模型有时还会先尝试旧参数, 再根据错误自我修正, 这是当前唯一可以马上处理的 Agent 易用性小问题。

3. Todo: 当前工作的执行清单

Agent 可用 TaskCreate、TaskGet、TaskList、TaskUpdate 管理任务、状态与依赖。清单可以从 transcript 中恢复, 因此 reload、compaction 和 session tree 变化后不会凭空丢失。成功的 Task Tool 不再在 transcript 中重复刷一行, 因为界面已经显示结果。

真实限制: 当前主要是 Agent 管理型 Todo。还没有完整的 /todo 或 /tasks 用户管理页, 用户不能直接在页面中新增、取消和重排; 代码中也没有真正的 reorder 字段。现在的显隐快捷键是 Ctrl+Shift+T, 还不是早期参考中的 Ctrl+T。

4. BTW: 主任务不停, 旁边问一句

`/btw` 问题 会在主 Agent 继续运行时, 使用当前已完成的上下文生成一次无工具回答。问题和回答不会写进主 transcript, 也不会进入主模型上下文。回答支持流式显示、取消、当前 session 历史、重启与 resume 后恢复、复制、清除, 以及用 `f` 把选中的问答显式提升为一个新 session。

真实限制: Pi 目前没有“由 Host 完整管理、但又完全不写 transcript”的公开旁路调用。BTW 因此直接调用当前 provider, 能继承 model 和 auth, 但不能继承完整 provider hooks、Host retry 与 transport 设置。

5. Permissions: 尽量不打断, 只防明显事故

普通读取、修改、测试、构建和工作区内明确删除默认直接执行。删除项目外明确目标时, 会询问是否只批准这一次。删除 `/`、用户 home、当前目录、Git worktree 根或其祖先时直接拒绝。常见 destructive Git discard 同样经过这层保护。

如果系统已经看出这是危险删除, 但目标仍含变量、替换、glob 或无法可靠解析, 就拒绝并要求 Agent 改写成明确命令, 而不是让用户批准一段含义不清的命令。子 Agent 的请求也回到根 session。

真实限制: 它是防手滑断路器, 不是 OS sandbox。任意程序、生成脚本、MCP 或第三方 Extension 内部的副作用不在完整保证内。

6. UI: 让以上能力像一个产品

UI Package 不只是样式包。它还负责临时页面的排队、权限界面抢占 BTW 后的恢复、输入草稿、footer、working row、Todo 和 roster 的一致恢复, 以及设置文件的安全写入。没有这层协调, 各插件单独“看起来不错”仍会在同时出现时打架。

| 是否能用、好用、可靠

对已经交付的范围，答案是肯定的；但要带上三个边界：它是本机预发布版，只认证一组精确环境，也没有覆盖所有未来能力。

1. 是否能用：可以

- 当前 Pi 配置已经加载本地 Aggregate Package。
- main 与 origin/main 同步，最近一批 UI 和 Agent 修复已经推送。
- Tool live call、reload、fresh resume 和 session resume 都保持紧凑样式。
- Agents、BTW、Permissions 和共享 Dialog 都经过真实 Pi PTY，而不只是 HTML mockup。

2. 是否好用：第一阶段已经达到

- 常用信息不会重复出现在 statusline、Todo、roster 和 transcript。
- 需要专注查看时进入同一种临时页面；结束后回到原草稿。
- 窄屏先删次要信息，身份和状态不会黏连或重叠。
- 权限默认安静，不把正常编码变成连续确认。
- 长结果不会靠全局展开来查看，降低历史很长时的卡顿风险。

3. 是否能出色完成指令：基础已经具备，完整闭环尚未具备

现在它可以把任务拆成 Todo、并行委托子 Agent、运行工具、在需要时保护危险命令，并让用户在不中断主任务的情况下提问。这已经覆盖了“认真完成一次 coding 任务”的主干。

但它尚不能依靠 Pi Stuff 自己做到跨 session 记住项目经验，也没有正式 Plan 审批、后台 shell 管理、Goal 自动续跑、定时 Loop、rewind、Web/MCP 连接目录。因此，长期、跨 session、需要外部系统的完整 workflow 仍依赖 Agent 临时组合普通 tools 或外部插件。

验证证据

项目	当前证据
自动测试	76 个测试文件, 453 个声明测试; 最新完整 gate 通过。
静态质量	格式、typecheck、Knip、generated files 和 public-safety 检查通过。
真实 Host	精确 Pi 0.83 源码构建、Package loader、源码包与解包后的 npm tarball 均有认证。
真实 TUI	100×32、64×28 以及 48/32/24 列宽矩阵覆盖; 包括中文、emoji、长路径和状态变化。
复杂路径	Agents execution matrix、nested permissions、BTW 并行、Tools resume 与 packed Aggregate 已验证。

认证边界: 当前支持口径是 Linux x64、精确的 post-tag Pi 0.83 源码、Bun 1.3.14、Node 24.16.0 和 npm 11.13.0。官方 `v0.83.0` tag 缺少 Live Thoughts 所需的公开 Markdown hook, 不能仅凭相同版本号视为兼容。

| 我们约定的，还有什么没做

这里必须分层。只有已经确认且可执行的内容才算当前欠账；方向研究、上游等待和明确排除不能混在一个“未完成”列表里。

A. 现在就应该补的三件事

事项	现在的问题	完成后的样子
归一 Beads 总账	用户要求所有决定进入 Beads，但未来路线主要散在研究文档和旧报告；Beads 仍留有旧 Hermes 选择，而后续方向已改为 Mem0。数据库中已关闭的两项 UI issue 与提交的 JSONL 快照也曾出现不同步。	用新的 canonical 记录明确 Mem0 取代 Hermes；把已确认路线拆成独立 issue、依赖和 acceptance；同步公开 JSONL。
Agent Tool 参数说明小修 ps-5cb.5.1	真实模型偶尔先使用旧参数或互斥参数，收到校验错误后才自行修正。能力没有坏，但会浪费一次尝试。	模型第一次就能正确启动单个或并行 Agent；错误形状仍被明确拒绝。
正式发布准备	七个 package 都是 0.0.0，有 9 个未消费 changeset；当前只能本地加载。	冻结第一版范围、消费 changeset、远端 CI 通过并从 npm 安装。是否立刻发布可由日用反馈决定。

B. 下一项已经确认的大能力：Mem0 长期记忆

跨 session 项目记忆已经确认属于默认 Pi Stuff。用户后来选择 Mem0，目标是本地存储、按项目隔离、可查看、可纠错、可删除、可关闭，不默认把记忆上传到外部服务。Hermes 只保留为原型失败时的回退参考。

目前仓库只有研究文档，没有 Memory Package、数据库、检索、/memory 或真实 Pi/Bun 原型。原先约定的可丢弃验证也还没有完成：

- 用约 20 条真实项目记忆验证写入、召回、修改和删除。
- 验证同名仓库不会串记忆，重启后仍能找回。
- 验证 100、1,000、10,000 条规模下的速度和结果质量。
- embedding 失败时退回关键词检索，Memory 故障时不能拖垮主 Agent。
- 确认默认零意外外传，并让每次写入和召回可见。

当前最自然的新能力

UI 第一阶段已经闭环，Memory 是下一项最值得做的产品能力。但第一步不是直接把大库搬进来，而是先做可丢弃的真实原型，验证本地存储、隔离、检索和故障退化。

C. 方向和 fork base 已讨论，但还没有正式实现

下面这些内容已经做过产品讨论或候选研究，但大多还没有独立 Bead 和正式排期。它们应写成“未来路线”，不能假装现在已经承诺立即交付。

能力	做完后你会看到什么	当前状态
Plan + 结构化提问	Agent 先只读调查，给出计划；需要选择时出现清楚的选项页，批准后再实施。	候选已研究；没有生产 package、批准流程或问题页面。
后台 shell / Monitor	测试、server 或监控任务可后台运行；能查看输出、停止和重新打开，不与子 Agent 混淆。	候选已研究；没有 runtime service、/ tasks 或统一运行视图。
Goal	给出明确完成条件，Agent 在安全上限内继续工作，并展示时间、轮次、token 和最近判断。	候选已选；没有 /goal、恢复或安全上限实现。
Loop / Cron	“十分钟后再次检查”或按计划唤醒当前 session；忙时只合并一次提醒。	候选已选；没有调度、到期、合并或管理页。
rewind / recovery	工作出错后恢复文件、对话或两者，并能撤销恢复。	候选已选；没有冲突保护、非 Git 路径或子 Agent 恢复。
Web / 文档读取	搜索并读取网页、GitHub、PDF 和下载文件，给出来源与明确失败原因。	候选已研究；没有正式 Package，也没有 URL、redirect、DNS、压缩和 PDF 限额。
MCP / 外部连接	按需连接外部 server，清楚显示访问范围、登录和降级原因。	候选已研究；没有信任、scope、OAuth、lazy catalog 或设置。
/doctor 与默认 Skills	能看到哪些能力已连接、哪些降级；常用流程有稳定 skills 入口。	只有产品意图，没有 runtime resource、页面或验收。

D. Todo 还有一个早期约定没有完整兑现

早期讨论希望用户能直接新增、取消和重排 Todo。当前实现只提供 Agent Tools 和只读清单投影，没有完整的用户管理页，也没有真正的 reorder。因此若继续坚持早期约定，需要单独决定并记录交互入口；这不是当前 Todo 的稳定性 bug，而是尚未实现的产品面。

E. 两项等待 Pi 上游，不是当前半成品

事项	为什么等待	当前可用替代
Tool 相邻分组 ps-5cb.7.1.1	Pi 0.83 没有公开的相邻 transcript grouping/transform 接口；不能靠改 session 数据或 fork Pi 绕过。	保持一项操作一行，使用 / tools 查看局部详情。
BTW 完整 Host 调用链 ps-5cb.3	Pi 没有完整 Host 管理且 transcript-free 的旁路调用接口。	当前 BTW 直接走 active provider，仍保持无工具、可取消和主对话隔离。

Beads 现在真实是什么状态

运行中的 Beads 数据库只有四个未关闭项：一个产品总账 epic、一个 Agent 参数易用性小修、一个 Tool 分组上游阻塞、一个 BTW 上游等待。换句话说，很多未来方向还没有从长篇研究转成真正可执行的项目任务；这正是报告中最需要补齐的管理问题。

| 明确不做，不算欠账

成熟不是把所有功能都塞进默认安装。以下边界已经明确，不能在未完成清单里反复出现。

不做什么	具体含义
不 fork Pi	Pi 保持 Host；Pi Stuff 只通过公开 API 和正常 Package 机制扩展。
不造第二套 Host / CLI / session / TUI	不把 Pi Stuff 变成另一个 coding-agent 平台。
不做浮动窗口	复杂交互统一用全宽、非浮动 Command Dialog；设置用 Pi 原生组件。
不做 LSP	不是默认、可选或发布 gate；需要者可另装独立 Pi Package。
不默认内建可交互浏览器	网页读取与外部连接可通过普通 tools、MCP 或独立 package 达到。
不内建跨 session Fleet / Agent Teams	当前 Agents 只服务一个主 Pi session，不扩张为团队或 workflow 平台。
不做 capability-specific statusline	Agent、Todo、BTW、Permission 和 Tool 状态留在自己的权威位置，不重复到 footer。
不保证重绘任意第三方 Tool	只保证内建 Tool、Pi Stuff 自有 Tool 和主动采用共同 renderer 的工具。
不复制旧代码或 Claude 代码	旧 jczhang02/pi-agent 与 Claude Code 只作能力和可观察行为参考，不能作为实现代码来源。

仍然有效的产品原则

1. 成熟 package 优先，但必须 owned fork。固定上游版本、commit 和许可证后纳入 monorepo，由 Pi Stuff 自己负责方向、UI、修复和发布。
2. 默认装上就能用。不要求用户逐个发现、启用和配置基础能力。
3. Claude Code 是体验参考，Pi 是技术边界。复现信息层级和操作感觉，不复制代码，也不违反 Pi 的公开能力。
4. 主界面以对话和输入为中心。先摘要，再进入细节，再提供动作；没有内容时不占行。
5. 同一状态只有一个权威位置。不做重复 dashboard。
6. 普通编码不打断。只有明确灾难性操作才拒绝或审批。
7. 所有产品决定进入 Beads。讨论必须用具体使用画面和人话，不靠术语逼用户选择。

| 建议的下一步

现在不应该再把精力平均分给十个未来能力。第一阶段 UI 已经可用，下一步应先把账本与日用小摩擦收干净，再进入 Memory。

第 0 步：把 Beads 变成可信总账

先记录 Mem0 取代 Hermes、Todo 直接管理面的真实状态，以及哪些未来方向只是候选、哪些已经正式承诺。为每个要做的能力建立独立 Bead、依赖、acceptance 和 out-of-scope，并同步数据库与提交的 JSONL 快照。

第 1 步：完成唯一可立即执行的现有小修

处理 ps-5cb.5.1，让模型第一次就能正确填写 Agent Tool 参数，同时继续拒绝含义冲突的调用。完成后再用真实模型跑一次单个和并行 Agent dogfood。

第 2 步：做 Mem0 可丢弃原型

只验证本地存储、项目隔离、检索质量、规模、重启、修改删除、embedding 故障回退和零意外外传。原型成功后再建立正式 Memory Package 和 /memory；失败就及时丢弃并重新评估，而不是让半成品进入 Suite。

第 3 步：根据真实日用决定 UI 1.1

持续使用现有版本，记录真正影响工作的摩擦。优先考虑 Todo 用户管理面、普通 transcript 的 diff/test/error 可读性，以及 Welcome/Statusline 的实际日用反馈；不要仅因为“还能美化”就继续扩大 UI 范围。

第 4 步：从未来路线中一次只选一条

Memory 之后，再根据日常价值选择 Plan/Ask、后台 shell/Monitor 或 rewind。Goal、Loop、Web 和 MCP 都需要额外安全、恢复或连接边界，应该在共同基础稳定后逐项进入，而不是同时 fork 八个 package。

第 5 步：形成第一版公开发布

当当前六项能力经过一段真实日用、Agent schema 小修完成、Beads 归一且远端 CI 稳定后，冻结范围、消费 9 个 changeset、提升版本并发布 npm。Memory 不必强行成为第一个公开版本的前置条件，但发布说明必须明确当前能力与限制。

顺序	目标	完成标志
0	Beads 与文档归一	Mem0、未来路线、Todo gap 和上游等待都有唯一可信记录。
1	Agent schema 小修	真实模型第一次正确启动单个和并行 Agent。
2	Mem0 prototype	隔离、质量、规模、重启、降级和零外传通过。
3	UI 1.1 dogfood	只修真实使用中高频、可复现的摩擦。
4	下一项生产能力	一次只推进一条，有正式 Bead 和真实 Host gate。
5	首个 npm release	版本、远端 CI、安装说明、支持矩阵和限制说明齐全。

报告结论

Pi Stuff 现在已经值得开始长期日用。接下来最重要的不是继续堆 UI，也不是同时引入所有成熟 package，而是先让 Beads 账本可信、修掉 Agent 参数的一次性摩擦，然后用真实原型决定 Mem0 是否能成为长期记忆底座。

| 依据与术语

A.1 本报告依据

- [Repository README](#): Package 形态、安装方式与未发布状态。
- [Aggregate README](#): 当前六项 Capability 与加载契约。
- [UI README](#)、[Tools README](#)、[Agents README](#)。
- [Todo README](#)、[BTW README](#)、[Permissions README](#)。
- [Compatibility](#): 精确 Pi Host 和工具链认证边界。
- [Mem0 fit research](#): Memory 方向与原型 gate。
- Beads ps-5cb : 产品总账; ps-5cb.5.1 : Agent schema; ps-5cb.7.1.1 : Tool grouping; ps-5cb.3 : BTW Host seam。
- 当前本机 Pi settings、package manifests、9 个 changeset, 以及最近真实 Host、PTY、resume 和 packed Aggregate 验证结果。

A.2 几个术语的人话解释

Host: Pi 本身。它拥有模型、主 Agent、session、编辑器、原生工具和 TUI。

Aggregate Package: 用户只配置这一个入口, 它按固定顺序载入整套 Pi Stuff。

Capability Package: 只负责一项清楚能力的包, 例如 Agents 或 BTW。

Command Dialog: 分隔线下方的全宽临时页面。它不是浮窗, 关闭后恢复原输入和界面。

owned fork: 从固定公开版本开始, 把来源、commit 和许可证记清楚后纳入 Pi Stuff; 后续产品方向和发布由 Pi Stuff 自己负责。

Beads: 项目的长期任务和决策账本。Todo 管当前 session 做什么, Beads 管项目长期答应了什么。

locally certified prerelease: 本机已经实际加载并通过测试, 但还没有 npm 正式版本, 也不承诺任意 Pi 0.83 环境都兼容。

A.3 审计口径

- 写“已经能用”, 必须在正式 packages/ 中存在、被 Aggregate 加载并通过真实 Host 认证。
- 写“方向已定”, 表示做过产品选择或 fork 研究, 不等于已经有生产代码。
- 写“等待上游”, 表示当前降级行为是有意且可用的, 不应通过 fork Pi 绕过。
- 写“明确不做”, 表示产品边界, 不计入欠账或完成率。